Text Sentiment Classification

Riccardo Rigoni, Filippo Selvatici, Guiv Farmanfarmaian, Maria Tsanta
Group: UnknownSentiment
Department of Computer Science, ETH Zurich, Switzerland

***Abstract*—We explore transformer-based models, specifically BERT variants, using different classification heads to perform sentiment analysis on a Twitter dataset. Our study investigates how innovative classification heads on top of BERT-based models can yield optimal performance. Moreover, we experiment with various pre-processing methods to enhance model accuracy. Finally, we set up a comprehensive framework, *BASH*, which enables automated ablation studies and incorporates both custom preprocessing steps and classification heads.**

I. Introduction

Social media platforms such as X (i.e. Twitter) play an increasingly important role in shaping our public sentiment and societal trends [1]. Gauging people's sentiments through their tweets could provide valuable insights into the reactions of certain demographics to events, products, or policies [2]. Sentiment classification consists of analysing text using Natural Language Processing (NLP) [3] to determine its sentiments. In this task, we aim to predict whether a tweet expresses a positive or a negative sentiment. Historically, the state-of-the art for such tasks were bidirectional Long Short-Term Memory [4] (bi-LSTM) networks coupled with a word vectorization. On the one hand, these models fail to model long-range dependencies [5]. On the other hand, the vectorizations may not capture the intricate nuances of language, especially when analysing tweets which contain misspelled words and slangs. Modern transformer-based architectures [5] tackle both of these issues.

In this work, we focus on architectures based on the Bidirectional Encoder Representations from Transformers - BERT [6]. These class of models provide high-quality embeddings which can then be fine-tuned for a specific task. After examining which BERT variant performs best, we fine-tune our model using a classification head.

Furthermore, we develop innovative classification heads, such as the Additive Attention Head that is on par with those used the SOTA in sentiment analysis and has a shorter training time. Additionally, we explore the benefits and drawbacks of using Kolmogorov-Arnold Networks (KANs) [7] as classifiers. This is to the best of our knowledge the first implementation of KANs on a sentiment analysis task. We also explore numerous pre-processing methods. Finally, we propose a comprehensive framework for sentiment analysis tasks based on BERT, called *BASH*. It allows for the comparison of custom pre-processing steps and classification heads.

II. Approach

Our sentiment analysis model consists of three main components: the pre-processing, the pre-trained base model and our custom classification heads.

We opted for BERT-based pre-trained models. Initially, we fed the pre-processed tweets into the tokenizer of the chosen BERT-based model. Finally, we developed creative classification heads that use the outputs of the transformer layers of the BERT-based models to predict the sentiment of a tweet.

Our ablation studies were performed on the reduced dataset consisting of 200K tweets (c.f. Appendix A1). We split those into training, validation, and test splits. In rare cases where results didn't differ fundamentally, we used K-Fold cross-validation. The detailed methodology can be found in Appendix A2.

Our final submission consisted of an ensemble, where each model was trained on the full dataset consisting of 2.5 million tweets (c.f. Appendix A1).

III. Pre-processing methods

Pre-processing tweets is crucial due to the presence of hashtags, slang, abbreviations, and misspellings. Below, we outline the most important pre-processing steps we applied. A comprehensive list of all the pre-processing methods we used is available in Appendix C.

A. Removing duplicate tweets

An explanatory data analysis revealed that some tweets appeared in both the positive and negative datasets, with different multiplicities in each class. We call them *duplicates*.

Duplicates with the same multiplicity were discarded. For the rest, we considered two approaches: either removing all duplicates or using a voting mechanism that groups the identical tweets, counts label occurrences and then determines the label with the highest count for each tweet. We conducted 3-fold cross-validation on the two duplicates removal strategies.

Results are summarized in Table I. Despite showcasing similar results, the voting mechanism reduces the standard deviation in the accuracy scores. Hence, we chose that approach.

Strategy	Val. Mean %	Std.	Test Mean %	Std.
Remove all	89.93	0.087	89.87	0.182
Voting	**89.94**	0.091	**89.98**	0.069

Table I: Duplicates removal strategy and related scores with BERTweet-base and default classification head (see sections IV and V).

B. Hashtags pre-processing

We considered multiple strategies when it came to dealing with hashtags. One strategy consisted of replicating the hashtags to emphasize their importance and of the sentiment they conveyed. However, this approach increased the maximum sequence length, rendering some tweets too long for our models.

Thus, we focused on a second strategy: splitting them into their constituent words. We used the `segment` function, from the `wordsegment` library [8] to segment each hashtag into its component words.

C. Replacing Slang

We compiled a comprehensive dictionary of slang terms and abbreviations to help translate informal language into standard phrases, thereby facilitating tokenization. Our dictionary is based on previous open source works [9] [10].

D. Advertisements

An analysis revealed the presence of *advertisement* tweets, like: *drama design protective decal skin sticker for amazon kindle e-book reader (version 1.0 this kindle skin is co ... <url>*. These are product descriptions from e-commerce platforms and are disproportionately classified as negative (over 100 as neg. vs 2 as pos. for Amazon ads). Since they also appear in the test set, we left them in the data corpus.

E. Pre-processing Discussion

We explored the optimal level of pre-processing to best highlight the determining characteristics of tweets. We defined three pre-processing levels (see Appendix D) and evaluated the same pre-trained BERT variant on each of them. Results are reported in Table II. The basic level consists of the same pre-processing used in BERTweet [11].

We conclude that the intermediate pre-processing level performs best and adopt it for all subsequent tasks. Thus, we have developed a pre-processing method that outperforms the commonly used techniques for Twitter-related tasks.

Finally, our understanding is that extensive text normalization obscures the full semantic nuance of tweets, as mentioned in [12].

Level	Validation Acc %	Test Acc %
Basic	89.84	**89.80**
Intermediate	**90.11**	89.72
Complete	84.51	85.01

Table II: Pre-processing levels and related scores with BERTweet-base with default classification head (see sections IV and V).

IV. Models and methods

To find the most appropriate architecture, we tested various models and conducted extensive ablation studies.

A. Transformer-based architectures

Transformer-based architectures use a self-attention mechanism [5] that processes the entire input sequence simultaneously, weighing the importance of different words regardless of their position. This captures both local patterns and long-range dependencies, as opposed to the previous SOTA bi-LSTMs. These models generate general-purpose embeddings for the input sequences, which are then fine-tuned on specific tasks.

We chose architectures based on BERT [6], since their performance has been shown to be SOTA for sentiment analysis tasks when compared to other open-source models [13] [14]. BERT architectures are designed to predict masked words in sentences, making them ideal for understanding the bidirectional context of a sentence [15]. As a result, they provide rich embeddings. Our study involved two steps: first, exploring various BERT model variants with a fixed classification head to identify the best performer, and second, implementing more advanced heads to improve accuracy, as detailed in section V.

B. Pre-trained Models

We experimented with the original BERT models. Then, we considered its variants such as RoBERTa (Robustly Optimized BERT Pre-Training Approach [16]) and its variant for sentiment analysis Twitter-roBERTa-base for Sentiment Analysis [17] . We evaluated both base and large versions of the models. Finally, we chose BERTweet [11], a model pre-trained on 750 million Tweets.

This provided us with the best embeddings as the model was already exposed to Twitter-specific language. We list some of our results in Table XII, and describe each pre-trained model as well as provide the full results in the Appendix H.

Model	Validation Acc. %	Test Acc. %	# Params
BERT-Base	86.79	87.63	110M
BERT-Large	87.11	88.08	340M
Twitter-roBERTa-base	87.84	88.36	110M
BERTweet-Base	89.69	**90.06**	110M
BERTweet-Large	**89.85**	**90.06**	340M

Table III: Performance Summary of Different BERT Variants

V. Classification Heads

We will now describe our various custom classification heads. Table IV summarizes their performances. In Appendix G, we present the full results of our ablation studies.

Head	Val. Acc. %	Test Acc. %	Time (h)	# Params
2-layer MLP	89.64	90.15	0.8 h	1.2 M
CLS-MLP	89.75	**90.18**	0.6 h	75 M
CLS-self-att.	89.72	90.05	1 h	2.4 M
ALL-self-att.	89.75	**90.22**	4 h	188 M
ALL-additive-att.	**89.78**	**90.17**	0.5 h	12 M
CLS-KAN	89.72	90.08	7 h	0.034 M

Table IV: Best scores of our classification heads with BERTweet-base on the reduced dataset (split 0.8/0.1/0.1). The last column is the total runtime (fine-tuning + inference) with 2 epochs. Parameters are computed with best hyper-parameters.

1) 2-layer MLP (default head): It consists of a 2-layer MLP with hidden size 768 (BERT models' embeddings size), dropout, and $tanh$ activation function after the first hidden layer. This head is implemented by the `RobertaForSequenceClassification` [18] class from the `transformers` library. It operates on the last layer's `CLS` embedding.

2) CLS-MLP: We extended the default classification head to incorporate the `CLS` embedding of the encoder's last n layers. The classifier then gains access to the various levels of semantic information captured by the different layers in the BERT architecture [19]. The model accuracy slightly increases from 90.15% to 90.18%.(Table IV).

3) CLS-self-attention: We tried using a self-attention mechanism [5] over the `CLS` embeddings. We denote by H be the $n \times 768$ matrix whose rows are the last n `CLS` embeddings. The mechanism computes $A = \text{MultiHead}(H, H, H)$:

$$\text{MultiHead}(H, H, H) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O \tag{1}$$

$$\text{head}_i = \text{Attention}(HW_i^Q, HW_i^K, HW_i^V) \tag{2}$$

Then, it averages the rows of A and feeds the result into the default classification head. The accuracy decreased to 90.05%.

4) ALL-self-attention: We extended the previous approach by applying the self-attention mechanism on all of the tokens' embeddings. In particular, we denote by L the number of tokens fed into the BERT model, and by $h_j \in \mathbb{R}^{768*n}$ the vector concatenating the embeddings of the j-th token in the last n encoder's layers, for $j \in [L]$. We assume that the `CLS` token has index 0. Moreover, we denote by H the $L \times (768 * n)$ matrix stacking the h_js. We compute

$${CLS_{new}}^\top = \text{MultiHead}(h_0^\top, H, H), \tag{3}$$

using the MultiHead() function defined above. CLS_{new} is then fed into the default classification head, with hidden dimension $768*n$. This head obtained the best test score (90.22 %, Table IV). However, this slight increase in performance didn't justify its significant training time (4x longer).

5) ALL-additive-attention: Self-attention has a quadratic complexity with respect to the input sequence length due to the projection matrices Q_i, K_i, V_i. Thus, we considered the additive attention [20], which exhibits a linear dependence on the input length.

Defining H and n as in the ALL-self-attention, our additive attention can be expressed as

$$a^\top = \text{Dropout}\left(softmax\left(tanh(HW_{att})v\right)\right)H, \tag{4}$$

where $W_{att} \in \mathbb{R}^{(768*L*n)\times att_size}$ and $v \in \mathbb{R}^{att_size}$ are trainable parameters.

Thus, $a^\top$ is convex combination of the rows in H, which is fed into a 2-layer MLP, with hidden dim. $768*n*L$. This head is on par with the SOTA and has a shorter training time.

6) CLS-KAN: Kolmogorov-Arnold Networks (KANs) [7] represent a recent breakthrough in machine learning, offering promising alternatives to traditional MLPs. While the latter have fixed activation functions on the nodes, the former have learnable activation functions on the edges. KANs can achieve comparable or better accuracy than much larger MLPs in data fitting[7]. Our KAN head consisting of a single layer can expressed as:

$$y^\phi = \sum_{q=0}^{2d}\left(c_q^\phi \cdot \text{spline}\left(\sum_{p=1}^{d}(a_{q,p}^\phi x_p + b_{q,p}^\phi)\right) + d_q^\phi\right), \tag{5}$$

where $d = 768$ is the input dimension, c_q^ϕ, d_q^ϕ, $a_{q,p}^\phi$, and $b_{q,p}^\phi$ are learnable parameters, spline() represents a spline function, and $\phi \in \{neg, pos\}$. The CLS-KAN classifier receives the last `CLS` embedding as input.

We use the `pykan` library, which provides a torch-compatible implementation of these networks. We could not fine-tune CLS-KAN on the full dataset[1], as the technology is still premature and its computational complexity not yet optimized. However, we find it remarkable that this head performs comparably with the others (90.08%) despite having significantly fewer parameters. This confirms the better neural scaling laws proven for these networks [7].

VI. Ensembles

To achieve our best performance, we leveraged ensemble methods and explored various strategies for combining classifier predictions. Initially, we experimented with a voting mechanism that assigns the most common label to each tweet. Ultimately, we opted for logistic regression on the positive and negative logits of the ensemble models. We set aside 5% of the training dataset to train this logistic regression model, which was then used to perform inference on the test set. Further implementation details are provided in the appendix M.

[1]The Estimated Time of Arrival (ETA) was more than 70 hours.

VII. Experiments

A. Baselines

We studied various baselines to justify our need for more advanced pre-trained transformer models. These baselines were fed the data with basic pre-processing.

1) Linear models: Our first choice was an ensemble of traditional linear models. We evaluated Bag-of-Words (BoW) and Term Frequency-Inverse Document Frequency (TF-IDF) for feature representation, alongside either Logistic Regression or Support Vector Machine (SVM) with a linear kernel for classification. The feature representations were invariant to word order, being based solely on the word frequencies in the tweets. In total, we obtained four different combinations of linear models which we combined into an ensemble. The full results and descriptions of our baseline experiments can be found in appendix J.

2) LSTM: We also evaluated bidirectional Long Short-Term Memory [4] networks (bi-LSTM) with GloVe (Global Vectors for Word Representation). GloVe embeddings provide dense vector representations for words based on co-occurrence statistics. Furthermore, we used the Twitter specific dataset glove.twitter.27B[2] to ensure relevance to our task. By using bi-LSTMs, we leveraged their ability to process sequences in both forward and backward directions, allowing the model to understand both contexts from preceding and succeeding words. This approach was the state-of-the-art (SOTA) [21] in natural language processing tasks before the advent of transformer-based models.

Moreover, we tried combining GloVe embeddings with either a GRU or a CNN (c.f. Appendix I for implementation details).

B. Results

We now explore the performance of our state-of-the-art ensemble of BERTweet models. We compare it both to the above-mentioned baseline and the advanced Twitter-roBERTa-base with our default classification head. We observe that using BERTweet as the pre-trained model, in combination with our Additive Attention classification head, results in noticeable improvements in performance. Finally, we test the architecture of the *Best Ensemble* model we provided in our submission. It consists of two BERTweet-large models (one with the CLS-MLP head and the other with the Additive Attention Head) and one BERTweet-base model with the Additive Attention Head.

The ensemble model was trained with a logistic regression on 5% of the reduced training set as explained previously in section VI. It is clear that the best individually performing models are BERTweet-Large (90.4%) and BERTweet-Base (90.14%), far outperforming both the baselines and Twitter-roBERTa-base. Combining the BERTweet models into an ensemble yields the optimal performance (90.7%).

[2]Link: https://nlp.stanford.edu/data/glove.twitter.27B.zip

C. Bert-based experiments setup

In order to better set up our experiment and allow for the easy reproduction of our results, we set up our own framework - Bert Analysis Sentiment Heads (BASH).
The framework is divided into three components: pre-processing, fine-tuning/inference and ensemble. These parts are designed to work together and are all controlled by configurations provided as input. The main benefits of the framework are:

- the ability to run combinations of '(pre-processed dataset, BERT model, classification head, hyperparameters)' by simply creating different config files. This enabled us to conduct numerous ablation studies, with the config controlling the experiment type, dataset subsampling, and whether to output logits and probabilities alongside predicted labels, etc.
- careful tracking of the experiments. For each run we save the outputs as well as a json summary file containing all the information we might need to access at a later time (e.g. the config and the validation scores).

See Appendix L and M for more details about the framework.

Model	**Validation Acc. %**	**Test Acc. %**
Linear Model Ensemble	80.75%	81.2
GloVe & bi-LSTM	83.25	83.05
GloVe & GRU	82.83	82.74
GloVe & CNN	79.79	79.41
Twitter-roBERTa-base	87.84	88.36
BERTweet-Base	89.81	90.14
BERTweet-Large	90.25	90.40
Best Ensemble	**90.43**	**90.7**

Table V: Performance of Baseline and SOTA Models

VIII. Summary

In this work, we studied how different models and classification heads can learn to classify the sentiment of a dataset of noisy tweets. In the end, our contributions are three-fold:

- We developed a framework for effectively conducting ablation studies combining different pre-processing strategies and sentiment classifiers;
- We implemented various innovative classification heads (such as the Transformer-based mechanisms, etc.) and showed that Additive Attention performs on a par with SOTA methods with a shorter ETA;
- We demonstrated that KAN-based classifiers perform similarly to MLPs with far less parameters.

Future work should focus on reducing the size of the CLS-KAN head (e.g. via an MLP) to enable their practical use alongside other classifiers. Another direction of work might revolve around identifying different classes of tweets such as advertisements, sarcastic tweets (studied in Appendix F) and others. We would then treat those tweets separately.

REFERENCES

[1] A. Ceron, L. Curini, S. M. Iacus, and G. Porro, "Public opinion on social media: How twitter represents the modern public sphere," *SAGE Open*, vol. 4, no. 1, 2014.

[2] V. K. K. M, P. Sunagar, and A. Kanavalli, "Analysis of sentiments in political-based tweets using machine learning techniques," in *2019 Global Conference for Advancement in Technology (GCAT)*, 2019, pp. 1–5.

[3] K. S. Jones, *Natural Language Processing: A Historical Review*. Dordrecht: Springer Netherlands, 1994, pp. 3–16. [Online]. Available: https://doi.org/10.1007/978-0-585-35958-8_1

[4] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," *CoRR*, vol. abs/1706.03762, 2017. [Online]. Available: http://arxiv.org/abs/1706.03762

[6] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: pre-training of deep bidirectional transformers for language understanding," *CoRR*, vol. abs/1810.04805, 2018. [Online]. Available: http://arxiv.org/abs/1810.04805

[7] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljačić, T. Y. Hou, and M. Tegmark, "Kan: Kolmogorov-arnold networks," 2024. [Online]. Available: https://arxiv.org/abs/2404.19756

[8] G. Jenkins, "wordsegment: English word segmentation, written in pure-python, and based on a trillion-word corpus." 2018.

[9] D. Effrosynidis, "Text-preprocessing-techniques/slang.txt at master · deffro/text-preprocessing-techniques," Apr 2019. [Online]. Available: https://github.com/Deffro/text-preprocessing-techniques/blob/master/slang.txt

[10] M. Mbaye, "Up-to-date list of slangs for text preprocessing," Feb 2020. [Online]. Available: https://www.kaggle.com/code/nmaguette/up-to-date-list-of-slangs-for-text-preprocessing

[11] D. Q. Nguyen, T. Vu, and A. Tuan Nguyen, "BERTweet: A pre-trained language model for English tweets," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, Q. Liu and D. Schlangen, Eds. Online: Association for Computational Linguistics, Oct. 2020, pp. 9–14. [Online]. Available: https://aclanthology.org/2020.emnlp-demos.2

[12] O. Owoputi, B. O'Connor, C. Dyer, K. Gimpel, N. Schneider, and N. A. Smith, "Improved part-of-speech tagging for online conversational text with word clusters," in *Human Language Technologies: Conference of the North American Chapter of the Association of Computational Linguistics, Proceedings, June 9-14, 2013, Westin Peachtree Plaza Hotel, Atlanta, Georgia, USA*. Association for Computational Linguistics, 2013, pp. 380–390.

[13] D. Cortiz, "Exploring transformers in emotion recognition: a comparison of bert, distillbert, roberta, xlnet and electra," 2021. [Online]. Available: https://arxiv.org/abs/2104.02041

[14] A. K. J. Z. Kate Pearce, Tiffany Zhan, "A comparative study of transformer-based language models on extractive question answering," 2021. [Online]. Available: https://arxiv.org/abs/2110.03142

[15] U. Naseem, I. Razzak, K. Musial, and M. Imran, "Transformer based deep intelligent contextual embedding for twitter sentiment analysis," *Future Generation Computer Systems*, vol. 113, pp. 58–69, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167739X2030306X

[16] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized BERT pretraining approach," *CoRR*, vol. abs/1907.11692, 2019. [Online]. Available: http://arxiv.org/abs/1907.11692

[17] F. Barbieri, J. Camacho-Collados, L. Neves, and L. Espinosa-Anke, "Tweeteval: Unified benchmark and comparative evaluation for tweet classification," 2020. [Online]. Available: https://arxiv.org/abs/2010.12421

[18] H. Face, "Robertaforsequenceclassification," accessed: 2024-08-01. [Online]. Available: https://huggingface.co/transformers/model_doc/roberta.html#transformers.RobertaForSequenceClassification

[19] K. Clark, U. Khandelwal, O. Levy, and C. D. Manning, "What does bert look at? an analysis of bert's attention," 2019. [Online]. Available: https://arxiv.org/abs/1906.04341

[20] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," 2016. [Online]. Available: https://arxiv.org/abs/1409.0473

[21] R. Mostafa, M. Mehedi, M. Alam, and A. Rasel, "Bidirectional lstm and nlp based sentiment analysis of tweets," in *Proceedings of the 14th International Conference on Soft Computing and Pattern Recognition (SoCPaR 2022)*, ser. Lecture Notes in Networks and Systems, A. Abraham, T. Hanne, N. Gandhi, P. Manghirmalani Mishra, A. Bajaj, and P. Siarry, Eds., vol. 648. Springer, Cham, 2023. [Online]. Available: https://doi.org/10.1007/978-3-031-27524-1_62

[22] L. Horne, M. Matti, P. Pourjafar, and Z. Wang, "GRUBERT: A GRU-based method to fuse BERT hidden layers for Twitter sentiment analysis," in *Proceedings of the 1st Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics and the 10th International Joint Conference on Natural Language Processing: Student Research Workshop*, B. Shmueli and Y. J. Huang, Eds. Suzhou, China: Association for Computational Linguistics, Dec. 2020, pp. 130–138. [Online]. Available: https://aclanthology.org/2020.aacl-srw.19

[23] J. Painter, H. Treharne, and D. Kanojia, "Utilizing weak supervision to create S3D: A sarcasm annotated dataset," in *Proceedings of the Fifth Workshop on Natural Language Processing and Computational Social Science (NLP+CSS)*, D. Bamman, D. Hovy, D. Jurgens, K. Keith, B. O'Connor,

and S. Volkova, Eds. Abu Dhabi, UAE: Association for Computational Linguistics, Nov. 2022, pp. 197–206. [Online]. Available: https://aclanthology.org/2022.nlpcss-1.22

[24] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, "Albert: A lite bert for self-supervised learning of language representations," 2020. [Online]. Available: https://arxiv.org/abs/1909.11942

[25] D. Q. Nguyen, T. Vu, and A. Tuan Nguyen, "BERTweet: A pre-trained language model for English tweets," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, Q. Liu and D. Schlangen, Eds. Online: Association for Computational Linguistics, Oct. 2020, pp. 9–14. [Online]. Available: https://aclanthology.org/2020.emnlp-demos.2

[26] J. Chung, Ç. Gülçehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," *CoRR*, vol. abs/1412.3555, 2014. [Online]. Available: http://arxiv.org/abs/1412.3555

APPENDIX

A. Methodology

1) Data: Our data consists of:

- `train_pos.txt` and `train_neg.txt` - a reduced dataset of 100K training tweets for each class
- `train_pos_full.txt` and `train_neg_full.txt` - a complete set of training tweets for each of the two classes, 1.25 M tweets per class.
- `test_data.txt` - the test set, i.e. the 10K tweets for which we had to predict the sentiment label in the Kaggle competition

Our reduced training dataset consists of a total of 200K tweets, whereas our full dataset consists of 2.5 Million training tweets. Our ablation studies were conducted on the reduced dataset where we used the same a training, validation and test split of 0.8/0.1/0.1. Our final run was trained on the full dataset where we reserved 5% of the data to train our ensemble. Note that for the ensemble on the reduced dataset *Best Ensemble* in our ablation studies, we set aside 5% of the training split of our reduced dataset.

The data labels were obtained using distant supervision where a tweet containing a smiley face ":)" was labeled as positive, whereas a tweet containing ":(" was labelled as negative.

2) Training methodology: For training the Bert-based models, we opted for the `AdamW` optimizer from the `transformers` package, initialized with `learning_rate` set to $2 * 10^{-5}$, `epsilon` of 10^{-8} and `weight_decay` set to 0.01. The optimizer was combined with a linear scheduler with warm-up. This choice aligns with is the original BERT model's optimization strategy. We used 0 warm-up steps for all of our ablation studies. However, both the ensemble in our submission and *Best Ensemble* used a warm-up period of 10% of the total training steps to stabilize initial training and prevent large gradient updates. We used a total of two epochs, as validation scores tended to decrease after that. The batch size depended on the model we chose and the capacity of the GPU at hand (whether we used the Euler Cluster or Google Colab). Our ablation studies were primarily conducted on the Euler cluster using a single NVIDIA V100 GPU. However, our final submission was ran on both the Euler Cluster and on Google Colab where we used either the L4 GPU or the NVIDIA A100 GPU.

Thanks to our framework, we can store the training state and the model parameters at the end of each epoch, allowing us to split the training in different jobs. This is very handy when the model requires more than 10 hours to train an epoch, thus reaching the wall-time limit on the Euler cluster if trained for two or more epochs.

B. Loss Function and Evaluation

The loss function we chose is the cross-entropy loss which is defined:

$$L(y_i, \hat{y}_i) = -\sum_{i=1}^{n} y_i \log(\hat{y}_i) \tag{6}$$

where y_i is the true label and $\hat{y}_i$ is the predicted probability of the true class. To evaluate our model, we used the accuracy score of its predicted labels on the validation and test split. This is also the score used to evaluate our submission on Kaggle.

C. Pre-processing steps

As mentioned, we considered various pre-processing methods for our dataset. Here we describe the approaches that we didn't mention in the report.

1) Removing stop words: Stop words, which include prepositions, pronouns, and auxiliary verbs, are words we filter out from our data because they are deemed insignificant. To do this, we use the `stopwords` function from the `nltk` library.

2) Removing repeated characters: We normalize the tweets by allowing the same letter to appear at most three times consecutively in the same word.

3) Spelling correction: Since tweets are often misspelled, we use the `SymSpell` function from the homonym library, to correct spelling. This way we can ensure that the misspelled version of a word and the correct version of it are the same for our model.

4) Lemmatization: Lemmatization in linguistics is the process of grouping together the inflected forms of a word so they can be analysed as a single item, identified by the word's lemma, or dictionary form. This can help flatten different words into one so that our model can process data more efficiently. To do this we used `WordNetLemmatizer` from the `nltk` library.

5) Replacing user and url tags: Following the idea that the '<' and '>' symbols might be misinterpreted by an NLP model [22], we replaced the < user> and <url> tags with xxuser and xxurl.

D. Pre-processing levels

The preprocessing steps were grouped into three levels: basic, intermediate, and complete.

- *basic:* voting mechanism;
- *intermediate: basic* + split hashtags + remove repeated characters;
- *complete: intermediate* + spelling correction + remove stop words + lemmatize tweets

E. Slang

As mentioned in Section III, we provided a comprehensive dictionary of slang terms and abbreviations. Our intuition was that using these mappings from slang words to standard vocabulary may help the models better understand

the tweets. We compare the Intermediate level of pre-processing with the Intermediate level with the slang pre-processing. We use the BERTweet base model with a default classification head. Our assumption before this experiment

Pre-processing Type	Validation Acc (%)	Test Acc (%)
Intermediate	90.11	89.72
Intermediate+Slang	89.77	90.16

Table VI: Accuracy results with and without slang pre-processing

was that BERTweet would be able to pick up the nuances of slangs, since it is pre-trained on hundreds of millions of tweets. However, this experiment demonstrates that mapping slangs to their corresponding words can still be helpful for BERTweet. One avenue for future work would be to test if this still holds for BERTweet-large, or whether that model would be able to pick up these nuances.

F. Sarcasm

Sarcasm detection remains an ongoing challenge in many NLP tasks. This is particularly challenging for us due to the distant supervision labeling of our tweets. Thus, a tweet containing a smiley face may actually exhibit a negative sentiment. In our attempt to combat sarcastic tweets in our dataset, we used a BERTweet base model fine-tuned on the sarcasm-annotated dataset (SAD)[23]. We performed inference on the different datasets (training and test), predicting the probability that the tweets were sarcastic. Once the model's output crossed a threshold of 0.9, we flipped its predicted label if the latter was positive. As a consequence, a tweet whose predicted label was positive would be relabelled as negative if its sarcasm score crossed the threshold of 0.9. However, upon closer examination of the dataset, many tweets crossing the threshold do not actually appear to be sarcastic. They were mainly fans seeking attention from their favourite celebrities or other users. An example of such a tweet is: `<user >#cantsayno can you do another justin bieber cover ? boyfriend maybe ? follow me :') 20`. Furthermore, the tweets labelled as sarcastic appeared as frequently in the positive and negative classes of the training data (0.77% of the positive tweets and 0.64 % of the negative tweets in the reduced dataset).

The poor performance of this sarcasm detection model could be explained by two factors. First, it has been trained on a dataset containing a balanced number of sarcastic and non-sarcastic tweets. Second, there may not actually be that many sarcastic tweets in our dataset.

G. Ablation studies classification heads

In this section, we present the ablation studies to determine the optimal hyperparameters for our classification heads. We used the data with intermediate pre-processing for all these runs.

First, we experimented with the following additional heads that we didn't further explore as their results weren't promising.

1) 2-layer MLP with layer normalization: This applies layer normalization after the first layer of the default head.

2) Single-layer Perceptron (SLP): This head consists of a single hidden layer with dropout

For all the following heads, we conducted extensive ablation studies.

3) CLS-MLP: The parameters we considered are the dropout probability and the number of last layers for the `CLS` embeddings.

Dropout	num layers	Validation Acc. %	Test Acc. %
0.1	4	89.68 %	89.93 %
0.2	4	89.60 %	90.06 %
0.3	4	86.63 %	90.11 %
0.4	4	89.76 %	90.05 %
0.1	3	89.63 %	89.99 %
0.4	8	89.64 %	90.14 %
0.4	12	89.62 %	90.14 %

Table VII: Ablation study for the CLS-MLP classification head on the reduced dataset (split 0.8/0.1/0.1).

As best hyper-parameters, we consider dropout: 0.4 and number of layers: 8.

4) CLS-self-attention: The parameters we considered are the number of last layers for the `CLS` embeddings, and the dropout probability of the self-attention head.

Dropout att.	num layers	Validation Acc. %	Test Acc. %
0.1	4	89.46 %	89.98 %
0.1	8	89.70 %	89.89 %
0.1	12	89.42 %	89.85 %
0.1	4	89.41 %	89.74 %
0.2	4	89.70 %	90.02 %
0.3	4	89.72 %	90.05 %
0.4	4	89.34 %	89.69 %

Table VIII: Ablation study for the CLS-self-attention classification head on the reduced dataset (split 0.8/0.1/0.1).

5) ALL-self-attention: The parameters we considered are the number of last layers and the number of heads for the self-attention mechanism. Due to the resource limitations on the Euler cluster (10GB of RAM), we could not consider more than the last 8 layers.

num heads	num layers	Validation Acc. %	Test Acc. %
8	4	89.54 %	90.17 %
8	8	89.82 %	90.09 %
12	4	89.86 %	90.14 %
12	8	89.98 %	90.22 %
16	4	89.73 %	90.17 %
16	8	89.62 %	89.96 %

Table IX: Ablation study for the ALL-self-attention classification head on the reduced dataset (split 0.8/0.1/0.1).

6) ALL-additive-attention: For the additive attention mechanism, we considered three hyper-parameters: the hidden size for the the query and keys, the number of layers, and the dropout probability.

hidden size	dropout	num layers	Val. acc. %	Test acc. %
256	0.3	2	89.67 %	89.99 %
256	0.3	4	89.67 %	90.14 %
256	0.3	6	89.60 %	90.05 %
256	0.3	8	89.53 %	90.22 %
256	0.3	12	89.85 %	90.07 %
256	0.1	12	89.65 %	90.00 %
256	0.15	12	89.7 %	90.12 %
256	0.2	12	89.86 %	89.99 %
256	0.3	12	89.67 %	90.14 %
128	0.3	4	89.58 %	90.07 %
256	0.3	4	89.67 %	90.14 %
512	0.3	4	89.78 %	90.18 %
768	0.3	4	89.76 %	90.12 %
1024	0.3	4	89.86 %	89.94 %

Table X: Ablation study for the ALL-additive-attention classification head on the reduced dataset (split 0.8/0.1/0.1).

7) CLS-KAN: We run the ablation on two hyper-parameters, namely the degree (k) of the polynomial in the splines, and the number of splines on each "edge" (grid).

k	grid	Validation Acc. %	Test Acc. %
2	3	89.58 %	90.07 %
3	3	89.72 %	90.08 %
3	5	89.97 %	89.65 %
4	5	90.02 %	89.60 %

Table XI: Ablation study for the CLS-KAN classification head on the reduced dataset (split 0.8/0.1/0.1).

H. Pre-trained BERT Variants

1) BERT-base: The BERT-base [6] model has been pre-trained on English text (overall about 3.3 billion words). For its pre-training, 15% of the words in the input sequence are randomly masked, and the model learns to predict these masked words based on the context provided by the other non-masked words in the sequence.

2) BERT-large: The BERT-large [6] model is similar to BERT-base, having had the same pre-training. They differ mainly in their size and their capacity, where BERT-large has twice as many layers, four additional attention heads and a large hidden size. BERT-large has a total of 340 million parameters compared to the 110 million parameters of BERT-base.

3) ALBERT: The ALBERT (A Lite BERT) [24] model is an optimized version of BERT designed to reduce memory consumption and improve efficiency. ALBERT employs two parameter-reduction techniques to lower memory consumption and increase the training speed of BERT: factorized embedding parameterization and cross-layer parameter sharing. On top of the BERT's pre-training strategy, ALBERT introduces a sentence-order prediction (SOP) task to improve coherence understanding in sentence pairs.

4) RoBERTa-base: Robustly Optimized BERT Approach [16] model has the same architecture as BERT-base but has been pre-trained differently. Its training strategy improves on BERT's Masked Language Modeling by dynamically changing the masking pattern applied to the training data. Instead of keeping the same masked words throughout an epoch, it changes which words are masked during different passes. This approach provides the model with more diverse contexts and helps it learn more robust representations. It also significantly increases the size of the training data.

5) RoBERTa-large: RoBERTa-large is a larger version of RoBERTa-base, using the BERT-large architecture.

6) Twitter-roBERTa-base: This model is part of the TweetEval [17] benchmark and has been pre-trained on Twitter data. It is a a roBERTa-base model trained on around 58M tweets and finetuned for sentiment analysis. TweetEval is a comprehensive evaluation framework for sentiment analysis and other tweet classification tasks, featuring datasets for various tasks such as emotion recognition, hate speech detection, and stance detection, among others.

7) BERTweet-base: This is has been introduced by [25]. It is the first public large-scale pre-trained language model for English Tweets. Its architecture is the same as BERT-base. It consists of 12 transformer layers, 768 hidden units, 12 attention heads, and approximately 110 million parameters. However, it has been trained using the RoBERTa pre-training procedure on a corpus of 850 million English tweets.

8) BERTweet-large: This is a larger version of BERTweet-base with the BERT-large [6] architecture, trained on a larger corpus of English tweets. It consists of consists of 24 transformer blocks, with 1024 hidden units per layer, 16 attention heads, and approximately 340 million parameters.

9) Ablation Studies: The following ablations studies were conducted on the data with intermediate pre-processing, while considering the default classification head for all models.

Model	Validation Scores %	Test Scores %	# Params
ALBERT (base)	86.24	87.20	12M
BERT-Base	86.79	87.63	110M
BERT-Large	87.11	88.08	340M
RoBERTa-Base	87.55	87.93	125M
RoBERTa-Large	88.42	88.86	355M
Twitter-roBERTa-base	87.84	88.36	110M
BERTweet	89.69	90.06	110M
BERTweet-Large	89.85	90.06	340M

Table XII: Performance Summary of Different BERT Variants

I. Baseline Models Description

Here we mention some of the models whose descriptions we omitted or briefly skimmed over in the report.

1) Linear Model Ensemble: The Linear Model Ensemble is composed of the following 4 vectorization and model combinations:

- BoW+Logistic Regression
- TF-IDF+Logistic Regression
- BoW+SGDClassifier
- TF-IDF+SGDClassifier

2) bi-LSTM: The architecture of our Bidirectional LSTM model consists of an embedding layer, a dropout layer, a bidirectional LSTM layer for sequence modeling, and a dense layer with $softmax$ activation for binary classification.

The model is compiled with categorical cross-entropy loss and the Adam optimizer.

3) GRU: Gated Recurrent Units (GRUs) [26] are more recent alternatives to Long Short-Term Memory networks (LSTMs). They simplify the architecture of LSTMs by combining the forget and input gates of LSTMs into a single update gate and merging the cell state and hidden state. This results in fewer parameters, making GRUs computationally more efficient and faster to train than LSTMs.

Our chosen GRU architecture we used consisted of an embedding layer, a dropout layer, batch normalization and a dense layer with $softmax$ activation for binary classification.

4) CNN: Convolutional Neural Networks (CNNs) have been increasingly employed for sentiment classification due to their ability to effectively capture local features and patterns in text data. By leveraging convolutional layers to scan through word sequences, CNNs can identify key phrases and contextual relationships that contribute to the overall sentiment of a tweet message.

Our chosen CNN architecture consists of an embedding layer, a 1D convolutional layer with ReLU activation for feature extraction, a max-pooling layer, a flatten layer, a dense layer with ReLU activation for learning complex patterns, a dropout layer, and a final dense layer with $softmax$ activation for binary classification.

Our model is compiled with categorical cross-entropy loss and the Adam optimizer.

J. Baseline Experiments Results

Model	Validation Acc. %	Test Acc. %
BoW & Logistic Regression	80.78 %	80.78%
TF-IDF & Logistic Regression	79.45 %	80.56 %
BoW & SVM	80 %	79.90%
TF-IDF & SVM	80.56 %	80.78%
Linear Model Ensemble	80.75%	81.2
LSTM	82.30%	82.53%
GloVe & LSTM	83.25%	83.05%
GRU	82.44%	82.46%
GloVe & GRU	82.83%	82.74%
CNN	78.74%	78.49%
GloVe & CNN	79.79%	79.41%

Table XIII: Baseline models accuracy with basic pre-processing on validation set and our own test set.

Here, we provide the results of all of our baseline experiment studies. The models were fed the data with basic pre-processing.

K. BASH framework

One of our main contributions was the development of a comprehensive framework *BASH* (Bert Analysis Sentiment Heads) to perform sentiment analysis, or any other downstream tasks using pre-trained BERT variants.

It allows for automated ablation studies on custom pre-processing steps and classification heads, defined in a config file. We provide the capability of fine-tuning a given model, performing cross-validation and running inference on a separate test set. Furthermore, we provide the capability of combining any combination of these models into an ensemble. We also provide multiple strategies to construct these ensembles. These steps are described in the following subsections.

The framework already provides a series of pre-processing steps which can be applied to the text in the desired order using a config file. The code can be further adapted to test the performance of any given combination of pre-processing steps.

Furthermore, we provide multiple classification heads that can be applied on top of the pre-trained model. Custom classification heads can be added and tested.

L. The framework: finetuning and inference

The component of the framework that runs the finetuning and inference can execute 4 different experiment types, controlled by important fields specified in the config. The experiment types are:

1) when fine-tuning = False: Simply runs inference on a pre-trained model. See figure 1.

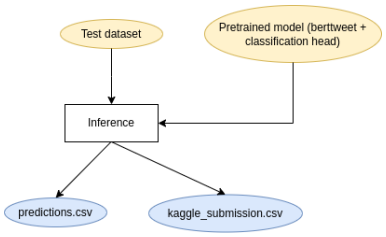

Figure 1

2) when fine-tuning = True and fine-tuning_config.cv_folds > 1: In this case we create a model using the pre-trained BERTweet model with a randomly initialized classification head, determined by the config. We then perform cross-validation using *finetuning_config.cv_folds* folds. Since cross-validation does not produce a single trained model, we don't run inference. See figure 2.

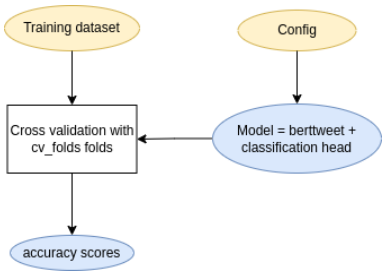

Figure 2

3) when finetuning = True and finetuning_config.cv_folds = 1 and ensemble_training = False: In this case we create a model using the pre-trained BERTweet model with a randomly initialized classification head determined by the config. We then fine-tune it and run inference with the trained model. For the finetuning, there is also a flag to split the training dataset into training+validation datasets, but we did not show it here. See figure 3.

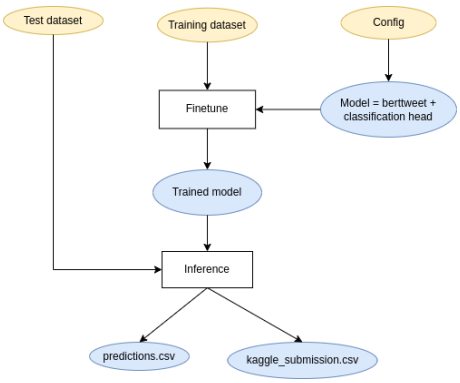

Figure 3

4) when finetuning = True and finetuning_config.cv_folds = 1 and ensemble_training = True: This case is very similar to the previous one, but the training dataset is initially split into training dataset and ensemble dataset. We run inference on the ensemble dataset to obtain a prediction file containing: the true label, the predicted label, the logits and probabilities values. The *ensemble_training_predictions.csv* is used for training the model used by the ensemble strategy. See figure 4.

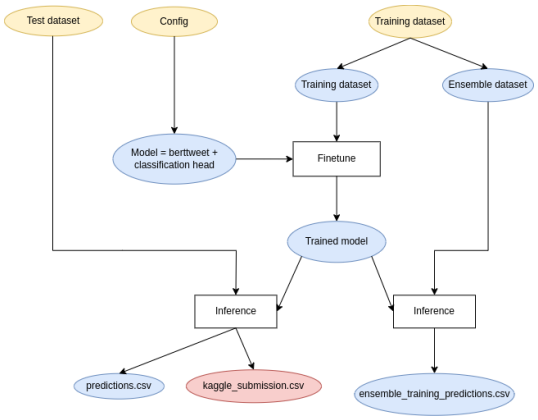

Figure 4

M. The framework: ensembles

The config for the ensemble specifies the ids of the model we want to ensemble together, as well as the strategy for the ensemble. Some strategies do not require training (e.g. majority voting), while others do (e.g. the *logistic_regression_on_logits* strategy). In case training is required, the ensemble models should be trained with experiments of type 4 because they leave out part of the training set for the ensemble training. Figures 5 and 6 depict the diagrams of the two types of ensembles, respectively.

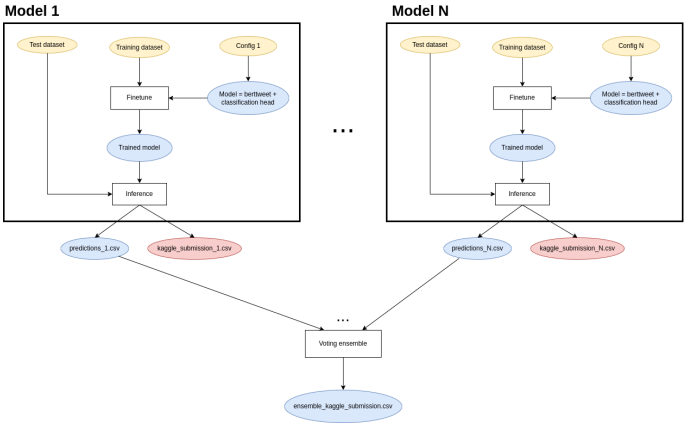

Figure 5

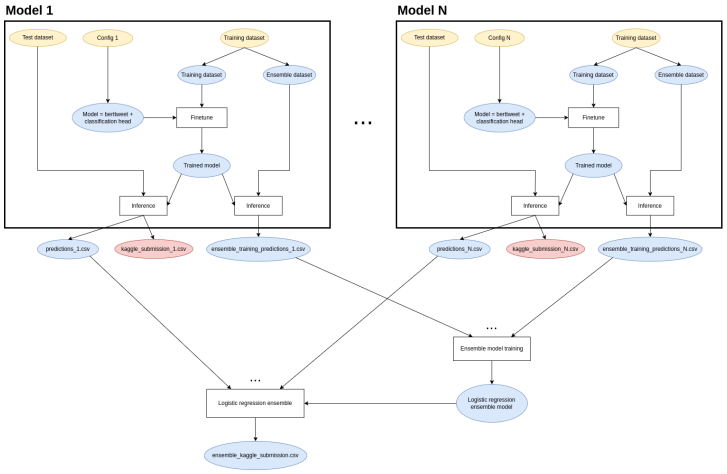
Model 1
Test dataset
Config 1
Training dataset
Training dataset
Ensemble dataset
Model = bertweet + classification head
Finetune
Trained model
Inference
Inference
predictions_1.csv
kaggle_submission_1.csv
ensemble_training_predictions_1.csv
...
Model N
Test dataset
Config N
Training dataset
Training dataset
Ensemble dataset
Model = bertweet + classification head
Finetune
Trained model
Inference
Inference
predictions_N.csv
kaggle_submission_N.csv
ensemble_training_predictions_N.csv
...
Ensemble model training
Logistic regression ensemble model
...
Logistic regression ensemble
ensemble_kaggle_submission.csv

Figure 6

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every semester paper, Bachelor's thesis, Master's thesis and any other degree paper undertaken during the course of studies, including the respective electronic versions.

Lecturers may also require a declaration of originality for other written papers compiled for their courses.

I hereby confirm that I am the sole author of the written work here enclosed and that I have compiled it in my own words. Parts excepted are corrections of form and content by the supervisor.

Title of work (in block letters):

TEXT SENTIMENT CLASSIFICATION

Authored by (in block letters):
For papers written by groups the names of all authors are required.

Name(s):	**First name(s):**
Farmanfarmaian	**Guiv**
Tsanta	**Maria**
Rigoni	**Riccardo**
Selvatici	**Filippo**

With my signature I confirm that
- I have committed none of the forms of plagiarism described in the 'Citation etiquette' information sheet.
- I have documented all methods, data and processes truthfully.
- I have not manipulated any data.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for plagiarism.

Place, date
Zurich, August 1st 2024

Signature(s)

For papers written by groups the names of all authors are required. Their signatures collectively guarantee the entire content of the written paper.